

Middleware

Middleware = Software-Schicht zwischen Anwendung und Betriebssystem/Netzwerk.

Aufgaben: - Abstraktion von Kommunikationsdetails - Infrastruktur für verteilte Anwendungen - APIs + Laufzeitumgebung

Typische Dienste: - Namensdienste - Transaktionen - Security - Persistenz - Nebenläufigkeit - Synchronisation

Entwicklung von Middleware

1. RPC Systeme
 2. Erweiterung um Transaktionen / TP-Monitore
 3. Message Broker (asynchrone Kommunikation)
 4. Object Broker (z. B. CORBA)
 5. Application Server
 6. Integration von Web-Technologien (REST)
-

Cross Cutting Concerns

Querschnittsfunktionen:

- Logging
- Security
- Transaktionen
- Monitoring
- Persistenz

Problem: Vermischung mit Business-Code.

Lösung: Container übernimmt Infrastruktur.

Komponentenbasierte Middleware

Trennung von:

- Business Logik
- Infrastruktur (Cross Cutting Concerns)

Container übernimmt:

- Kommunikation
 - Transaktionen
 - Security
 - Lifecycle Management
 - Thread Management
-

Container Architektur

Client → Proxy → Container → Bean

Container führt aus: - Cross Cutting Concerns - Pipeline Verarbeitung - Business Methode

Execution Pipeline

Request Verarbeitung:

1. Request Matching
 2. Request Filter
 3. Interceptors
 4. Business Methode
 5. Response Filter
 6. Response Verarbeitung
-

Thread-basierte Middleware

Server nutzt Worker Threads.

Ablauf:

Client Request → Listener Thread → Worker Threads → Business Code

Eigenschaften:

- parallele Verarbeitung
- Thread Pools

- Skalierbarkeit
-

Event-driven Middleware

Moderne Server nutzen:

- Event Loop
- Non-blocking I/O
- Reactor Pattern

Beispiele: - Node.js - NGINX

Langlaufende Aufgaben → Thread Pool.

Green Threads / Virtual Threads

Eigenschaften:

- leichtgewichtige Threads
 - viele Threads möglich
 - Runtime verwaltet Scheduling
-

Komponenten (Beans)

Komponente = Business Objekt das vom Container verwaltet wird.

Container übernimmt:

- Lifecycle
 - Dependency Injection
 - Security
 - Threading
-

Dependency Injection (DI)

Prinzip: Inversion of Control.

Framework erzeugt Objekte und injiziert sie automatisch.

Beispiele:

- DAO Objekte
 - Datenbankverbindungen
 - Services
-

Lifecycle einer Komponente

1. Instanz erzeugen
 2. Dependency Injection
 3. @PostConstruct
 4. Nutzung
 5. @PreDestroy
-

Interceptors

Interceptor = Methode vor/nach Business Methode.

Nutzen:

- Logging
- Security
- Monitoring
- Transaktionen

Typische Annotationen:

- @AroundInvoke
 - @PostConstruct
 - @PreDestroy
-

Filter

Filter arbeiten auf Request/Response Ebene.

Typische Aufgaben:

- Authentication
- Input Validation
- Logging
- Header Manipulation

Komponentenframeworks

- Jakarta EE
 - Spring / Spring Boot
 - .NET
 - Node.js Frameworks
-

Jakarta EE Architektur

Typische Module:

- EJB Container
 - Servlet Container
 - JPA
 - JMS
 - JAX-RS (REST)
 - JAX-WS (SOAP)
 - JNDI
-

Enterprise Java Beans (EJB)

Arten:

1. Session Beans
 2. Message Beans
 3. Entity Beans
-

Stateless Session Beans

Eigenschaften:

- kein Client Zustand
 - Methoden unabhängig
 - hohe Skalierbarkeit
 - Instanz Pooling
-

Stateful Session Beans

Eigenschaften:

- Client spezifischer Zustand
 - Session über mehrere Methodenaufrufe
 - Aktivierung / Passivierung möglich
-

Passivierung / Aktivierung

Passivierung: Bean Zustand wird gespeichert.

Aktivierung: Bean Zustand wird wieder geladen.

Ziel: Speicher sparen.

Entity Beans / JPA

ORM Technologie.

Entities:

- persistente Objekte
- Abbildung auf Datenbanktabellen

Operationen:

- persist()
 - find()
 - merge()
 - remove()
-

Persistenz Kontext

EntityManager verwaltet:

- geladene Entities
- Cache von Datenbankobjekten

Entities:

- Managed

- Detached
-

Wichtige Annotationen

Jakarta EE:

- `@Stateless`
- `@Stateful`
- `@Inject`
- `@Entity`
- `@PersistenceContext`

Spring:

- `@Component`
 - `@Service`
 - `@Repository`
 - `@Controller`
-

Klausur Kernthemen

- Middleware Architektur
- Container Konzept
- Execution Pipelines
- Dependency Injection
- Interceptors & Filter
- EJB Typen (Stateless vs Stateful)